

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página Web Productora 2 (SCRUM-20)

1. CONTEXTO

Se requiere una presencia web para una productora audiovisual/creativa. El stack está explícitamente restringido a HTML5 + CSS + JS puro (sin frameworks de backend, sin Node.js, sin bases de datos). El formulario de contacto debe enviar correos sin almacenar datos. El equipo de diseño aún no entrega assets finales. El sitio es un MVP de presencia digital con 4 secciones: Home, Servicios, Quiénes Somos y Contacto.

2. DECISIÓN ARQUITECTÓNICA

Arquitectura elegida: Sitio Estático Multi-Sección (Single-Page Application sin framework)

- Un único archivo `index.html` con 4 secciones ancladas mediante `id` y navegación por scroll suave. NO se crean 4 archivos HTML separados.
- CSS dividido en: `reset/base`, `layout`, `components` y `responsive` dentro de un único `styles.css` para minimizar requests HTTP.
- JS dividido en: `main.js` (navegación, scroll, animaciones) y `contact.js` (validación de formulario + integración con servicio de email externo como Formspree o EmailJS, que son servicios gratuitos compatibles con HTML estático).
- Para el envío de correos se utilizará **Formspree.io** (endpoint externo gratuito) o **EmailJS** vía CDN, eliminando la necesidad de backend propio. Esto es 100% compatible con el requerimiento de 'no almacenar datos en base de datos'.

3. JUSTIFICACIÓN BASADA EN YAGNI Y KISS

- **¿Por qué NO microservicios ni backend propio?** El requerimiento explícita 'HTML5, CSS y JS solamente'. Un backend Node.js/PHP sería BDUF puro. Formspree resuelve el envío de email sin una sola línea de código servidor.
- **¿Por qué NO múltiples archivos HTML?** Una SPA de una sola página reduce la complejidad de navegación, elimina recargas de página, mejora el tiempo de carga percibido y simplifica el despliegue a cualquier hosting estático (Netlify, GitHub Pages, cPanel).
- **¿Por qué NO un framework CSS (Bootstrap/Tailwind)?** El requerimiento pide CSS puro. Agregar un framework CDN externo añade dependencia de terceros y peso innecesario para un sitio de 4 secciones.
- **¿Por qué NO CMS integrado ahora?** El requerimiento funcional excluye explícitamente 'integración con sistemas de gestión internos'. La mención de CMS en architectural_requirements es aspiracional y contradice el stack aprobado. YAGNI: se diseña para que el contenido sea fácilmente editable en HTML directamente, preparando la estructura semántica para una futura migración a CMS sin refactoring mayor.
- **Imágenes:** Se usan URLs de `https://placeholder.co` y `https://picsum.photos` como mandato explícito del requerimiento. Cero archivos binarios locales.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (Bajo):** Formspree tiene límite de 50 envíos/mes en plan gratuito. Mitigación: documentar en README que se debe actualizar a plan pago antes del lanzamiento.
- **Riesgo 2 (Medio):** Sin backend, la protección anti-XSS e inyección SQL recae 100% en la validación client-side (JS) y en las políticas del servicio de email externo. Mitigación: sanitización estricta de inputs con JS antes del envío + Content-Security-Policy headers configurados en el servidor de hosting.
- **Riesgo 3 (Bajo):** El multi-idioma (ES/EN) mencionado en reglas de negocio no puede implementarse con HTML/CSS/JS puro de forma elegante sin un sistema i18n. Mitigación: se implementa un toggle de idioma básico

con JS que intercambia atributos `data-lang` en el DOM. Suficiente para MVP.

- **Riesgo 4 (Bajo):** CDN y pruebas de carga son responsabilidades de infraestructura/hosting, fuera del alcance del código entregado. Se documenta en README.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- css/responsive.css | Estado: APROBADO | Iteraciones: 1
- js/main.js | Estado: APROBADO | Iteraciones: 1
- js/contact.js | Estado: APROBADO | Iteraciones: 1
- assets/icons/favicon.svg | Estado: SKIPPED | Iteraciones: 0
- README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 81,012

Tokens de salida (output): 31,169

Total tokens: 112,181

Horas-hombre estimadas: ~10.8 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: 4,167 input | 1,473 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: 14,144 input | 8,998 output | 6 llamadas

FRONTEND-DESARROLLO: 20,485 input | 20,476 output | 6 llamadas

QA-VALIDACION: 42,216 input | 222 output | 6 llamadas